

CONTROLLER ROUTER

Deployment Guide

Package the library and optional UNO Q runtime,
with safe upgrades and recovery.

VIRTUALGLOVE / R.O.B. VISION / MAKER CONTROLLERS

Current source edition | 27 September 2026

This guide covers packaging and deploying Controller Router inside your own controller project. The Integration Guide describes the design; the Technical Reference defines the APIs.

Install the console library

The repository's `pyproject.toml` packages `router_shared` as `maker-controller-router`. It declares Python 3.10 or newer. From a local checkout, install it in your project's virtual environment:

```
python3 -m venv .venv
. .venv/bin/activate
python3 -m pip install .
```

The library has no third-party Python runtime dependencies. Live routing requires Linux input devices, uinput permission, EmulationStation mappings for ordinary pads, and RetroArch. A macOS development machine can inspect and test the API but cannot create Linux uinput outputs.

The wheel is Python-only and does not need a CPU-specific build. It does not install service units, a receiver, game launch hooks, a web setup interface, or UNO Q firmware. Package those integration assets in your project.

Package a complete integration

1. Include the shared library through your dependency package or a maintained vendored copy.
2. Install your receiver and its stable uinput device definition.
3. Install any custom source descriptor and set `CONTROLLER_ROUTER_SOURCES_FILE` for the Router process.
4. Choose persistent assignment, backup, and socket paths appropriate to the console.
5. Import known EmulationStation mappings and add your custom source without deleting saved assignments.
6. Start `ControllerRouterDevice` with explicit platform configuration paths.
7. Expose bounded, authenticated setup operations through your project's service.
8. Apply the merged output mapping to RetroArch and test the actual game.

Provide service startup, shutdown, watchdog, permissions, and logs in your installer. Keep personal configuration separate from replaceable software. Avoid creating a second competing Router process when another compatible integration already owns the merged outputs.

Router recognizes RetroPie, Batocera, and Recalbox configuration conventions. That recognition alone does not validate your receiver, emulator wrapper, or installer on every board and version.

Protect console configuration

Finish games before installing or reconfiguring virtual controllers. Close EmulationStation on RetroPie before recreating virtual joysticks; its input manager can fail during device replacement.

Router's atomic writes preserve existing permissions and ownership. When writing as root under `/opt/retropie/configs/`, the writer assigns `pi:pi`. Use that helper for managed writes rather than replacing a file through an unrelated root-owned temporary file.

Route mapping changes through the Router integration. Receivers and game-specific wrappers must not independently overwrite the same player indexes. Decide which platform files your adapter owns and document that boundary before deploying it.

Optional UNO Q runtime

The `uno_portal` directory supplies the browser chooser, host broker, product lifecycle service, lease coordinator, Matrix scheduler, and App Lab sketch. It is separate from the console wheel.

The current installer expects the `arduino` account with UID 1000, App Lab, Docker Compose, user services, and the existing registered product layout. It is designed for the current VirtualGlove and R.O.B. Vision adapters. A new app must extend that registration; this is not a generic install command for an arbitrary third product.

To add your app:

1. Add a fixed app ID and product path to the portal's registries and validation allowlists.
2. Define its browser port and local health endpoint.
3. Define how its authenticated game-session state becomes a live-game boolean.
4. Register its manifest path and include the display client in its runtime.
5. Add lease enforcement to its input producer and receiver.
6. Add the product to lifecycle startup and installer backup/upgrade handling.
7. Add its chooser entry and direct-visit selection behavior.
8. Extend tests before enabling it on a UNO Q.

Keep Router as the sole Matrix firmware owner. Product installation and startup should not flash a competing sketch. The shared Matrix's first App Lab build can take several minutes; keep progress visible and explain the wait to users.

Upgrade and rollback design

Store pairing credentials, game registries, and player assignments outside software replacement paths. Back up the current software before changing services. Verify new service and Matrix health; restore previous software if activation fails.

Keep a newer compatible shared Router when a product bundles an older copy, while still registering the new product. A newly installed controller must not disappear merely because shared code did not need upgrading.

Assignment rollback is separate from software rollback. RouterStore saves a `.previous` snapshot and requires a current revision for restoration. Your UI should explain which settings are being restored.

Existing integrations as deployment examples

R.O.B. Vision installs the shared console service and a fixed Buddy source descriptor, then keeps Buddy on Player 2 through its adapter. VirtualGlove vendors the same engine behind its signed console API. Both bundle `uno_portal` so their UNO Q installers register their services with the same Matrix owner.

Read the implementation before copying service paths or product assumptions:

- [R.O.B. Vision installer](#)
- [R.O.B. Vision Router setup](#)
- [VirtualGlove UNO Q installer](#)
- [Shared UNO installer](#)

Release checklist

Test a clean install, repeat upgrade, preserved assignments, failed-start rollback, input release on disconnect, and live-game configuration rejection. For UNO Q, also test no-selection reboot, automatic game selection, missing or expired leases, display expiry, and a newly registered app alongside a newer Router.

Record actual platform and device results. Keep build compatibility, mocked tests, and physical-device tests distinct in release notes.